

《编译原理》 Lab 3 : 中间代码生成 (21/5/2026)

姓名 / 学号 : 林武文 / 3230300081 专业 : 计算机科学与技术

Lab Introduction (ZERO IR)

Implemented IR code generation from AST into executable Zero IR.

Main Idea : connecting front-end & back-end through an IR

SysY → Lexer → AST → Semantic/Type → IR Translator → Zero IR → Interpreter

- IRTranslator later traverses the AST and emits Zero IR instructions

Lab Approach / 试验思路

1. Expression Translation

Each expression is translated into code that stores its result in a temporary variable or target place, e.g. : `a = b + 1`

```
T0 = b; T1 = #1; a = T0 + T1
```

e.g. : `translateExp` dispatches different AST expression nodes (`LVal` , `IntConst` ,...)

```
IR::Code IRTranslator::translateBinaryExp(AST::BinaryExpPtr node,
                                          const std::string& place) {
    auto left_place = new_temp(); auto right_place = new_temp();// translate
    auto left_ir = translateExp(node->left, left_place);
    auto right_ir = translateExp(node->right, right_place);
    append_code(ir, left_ir); append_code(ir, right_ir); //SNIPPED
```

- follows the idea that AST is tree-structured (whereas IR is linear) → each child expression must be translated first, then parent expression combines their result

2. Function Translation & Function Calls

Generate a `FUNCTION` instruction + emit `PARAM` instructions for each parameter, followed by the translated function body :

```

IR::Code IRTranslator::translateFuncDef(AST::FuncDefPtr node) {
    IR::Code ir; // Emit function entry label ↓
    ir.push_back(IR::Function::create(node->name));
    for (auto& param : node->params) { //register param in order
        std::string pname = ir_name(param->symbol, param->ident);
        ir.push_back(IR::Param::create(pname));}
    auto block_ir = translate(node->block); //trans funct body after
    append_code(ir, block_ir); // SNIP (below handles the void funct too)
}

```

For function calls, arguments are evaluated first, then passed using ARG + emit CALL

```

for (auto& arg : node->args) { //eval args first (left-right)
    auto arg_place = new_temp();
    auto arg_ir = translateExp(arg, arg_place);
    append_code(ir, arg_ir);
    arg_places.push_back(arg_place); } //SNIP
if (!place.empty()) { // Emit CALL, store return value if needed
    ir.push_back(IR::Call::create(place, node->name)); } else {
    ir.push_back(IR::Call::create(node->name));}
}

```

3. Control Flow Translation

Used labels and jumps, if and while converted into LABEL , IF , GOTO

4. Array Translation

Represented as memory blocks, use DEC to allocate continuous memory, array access via calculating byte offset + load/store through memory.

```

int stride = 1; for (int j = i + 1; j < (int)dims.size(); j++) {
    stride *= dims[j]; }

```

↑ computes linear index using array dimensions from ArrayType , ↓ final byte offset

```

ir.push_back(IR::BinaryImm::create(byte_offset, offset_elems, BinaryOp::Mul, 4));
ir.push_back(IR::Binary::create(addr_place, base, BinaryOp::Add, byte_offset));

```

5. Global Variables & Global Arrays

Must be emitted before all functions using the GLOBAL instruction. In translateCompUnit , iterate all VarDecl nodes and emit global IR declarations before translating function :

```

IR::Code IRTranslator::translateCompUnit(AST::CompUnitPtr node) {
    for (auto& unit : node->units) { // 1. Emit global variables first
        if (auto decl = std::dynamic_pointer_cast<AST::VarDecl>(unit)) {
            auto global_ir = translateGlobalVarDecl(decl);
            append_code(ir, global_ir);
        }
        for (auto& unit : node->units) { // 2. Then emit functions
            if (!std::dynamic_pointer_cast<AST::VarDecl>(unit)) {
                auto unit_ir = translate(unit);
                append_code(ir, unit_ir); // The rest is SNIPPED
            }
        }
    }
}

```

→ to access, compiler loads the address using `&label` then load/store to the address

6. Variable Scope Handling

Use `unique_name` (semantic A.) to map identical identifier to unique IR names.

```
std::string name = ir_name(node->symbol, node->ident);
```

Difficulties and Highlights / 难点与亮点

(-) **Array Initialization Aligning** : Nested array init was complex than simple flattening

e.g.: `int a[2][3][4] = {1, 2, 3, 4, {1}, {5}};`

- Braced sub-list `{5}` should initialize 1 sub-aggregate and zero the remaining elements, cannot use simple flattening strategy

→ calculate the sub-aggregate size based on depth & array dimension

(+) **Zero IR Execution** : not only has valid syntax but also executable by interpreter

- Final compiler supports recursion, loops, simple + complex logic, arrays, global variables, array parameters, and simple programs (sorting, Dijkstra, Sudoku)

Reflection / 心得体会

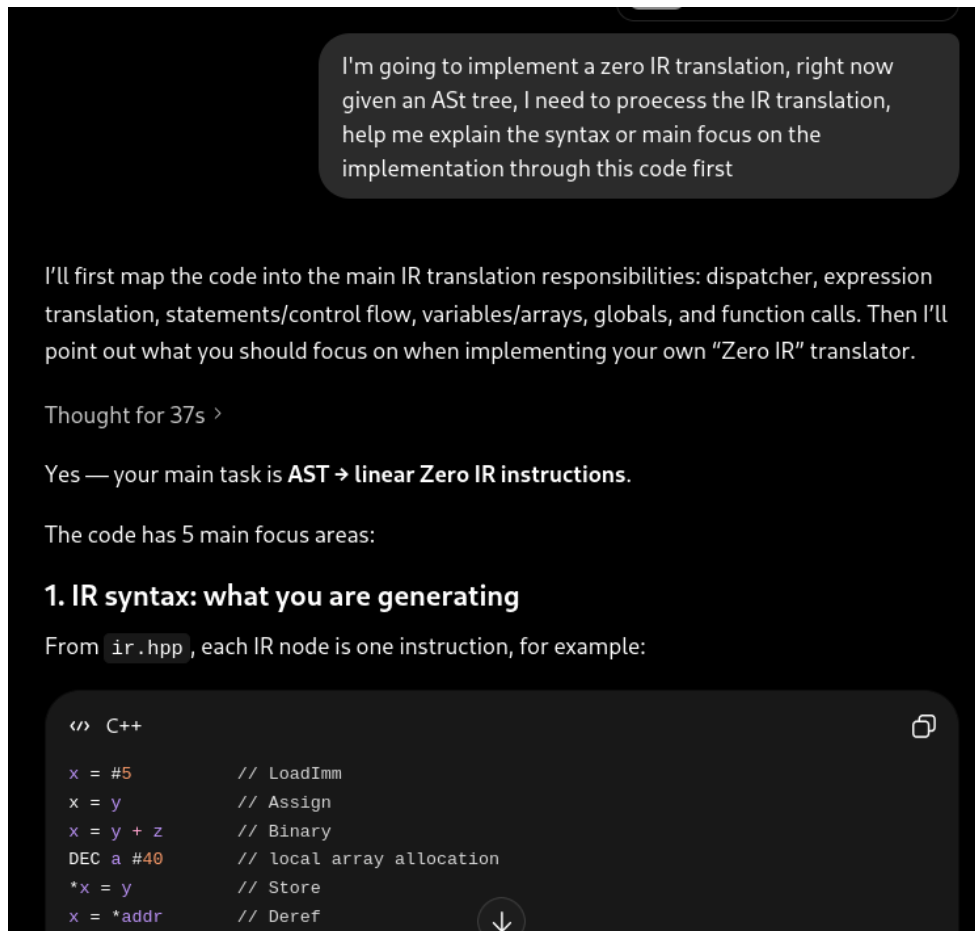
Through this lab, I learned how IR was generated from AST. Where IR generation requires thinking about data flow and control flow at the same time. Some details are essential for correctness such as variable scope, array layout, function parameter passing & global variable addressing. By debugging the generated IR with the interpreter, I also implement what I learned from class. Overall, this lab helped me connect front-end semantic with back-end code representation.

Additional Content / 附加内容

借鉴声明 : I didn't have any reference while doing this lab

AI 工具使用记录 : I only use ChatGPT (GenAI) for assist during the lab

For each code, the first step is to ask the basic syntax and guidance on key files



- While also assists in explaining the core concepts for each file along with syntax explanation

2. Dispatcher: choose translation function by AST type

This part:

```
</> C++  
IR::Code IRTranslator::translate(AST::NodePtr node)
```

checks the runtime AST type using `dynamic_pointer_cast`, then calls:

```
</> C++  
  
translateCompUnit(...)  
translateFuncDef(...)  
translateBlock(...)  
translateVarDecl(...)  
translateAssignStmt(...)  
...
```

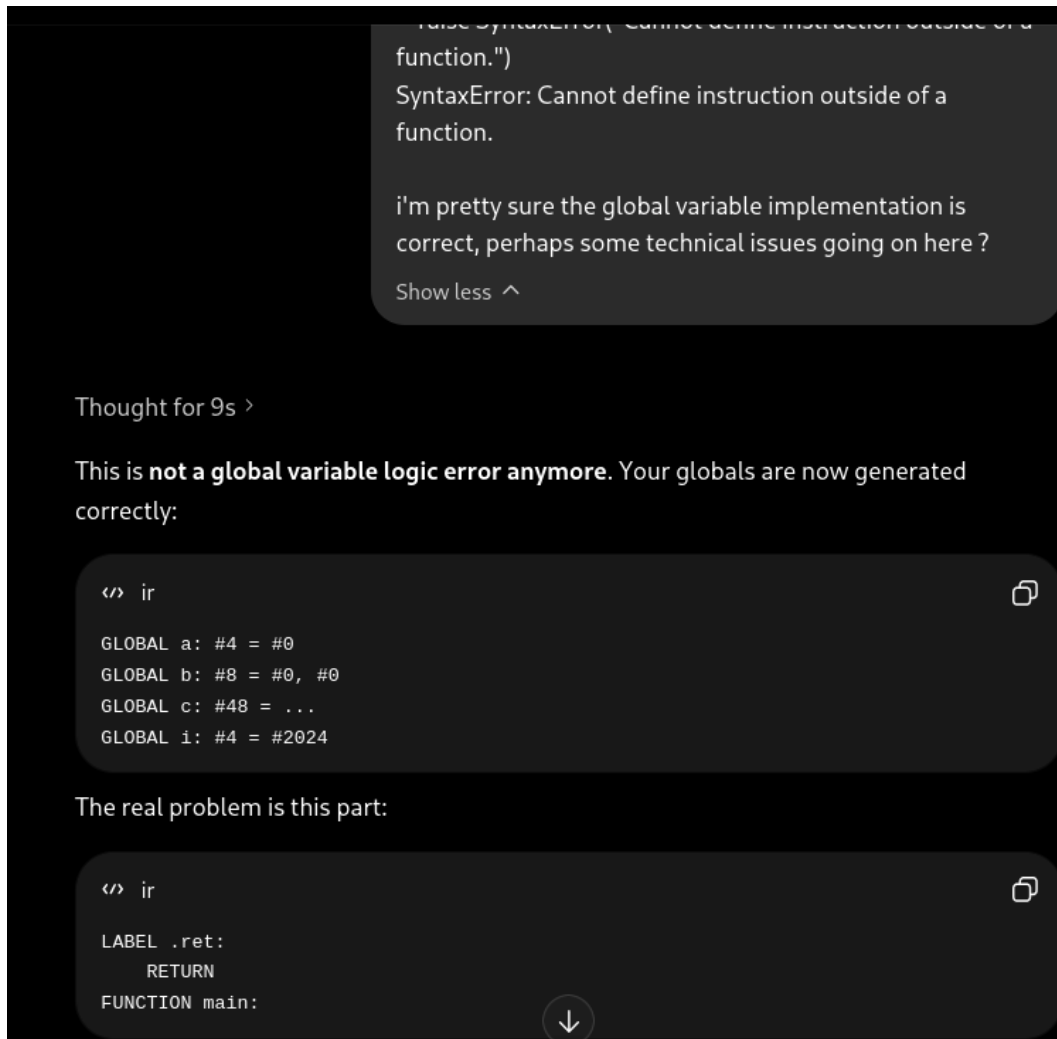
So every AST node type needs a matching `translateXXX`.

3. Expression translation uses `place`

Important pattern:

```
</> C++  
  
translateExp(node, place)
```

For some code error with minor tweaks, I sent the error along with the corresponding files for it to search the error with explanation :



During testing phase :

1. Since the test cases vary, some test cases might slipped out so I tend to ask ChatGPT about any missing test cases implementation

```
steps
tests/lab3/while_if_test1.sy      Accepted
2610 steps
tests/lab3/while_if_test2.sy      Accepted    90
steps
tests/lab3/while_test1.sy         Accepted    689
steps
```

looks like I missed some implementation (array, and global variables are not implemented yet), but I think I missed the unary operator as well

Show less ^

Thought for a couple of seconds >

Good progress. Your **control flow, function calls, recursion, while, if, short-circuit** are mostly working now. The remaining failures are mainly in these groups:

1. arrays
2. global variables / global arrays
3. array parameters
4. duplicate local variable names / scope
5. unary operator



1. First fix: unary_op.sy

This is probably the easiest runtime error. P

